LAPORAN PEMROGRAMAN WEB: TATA CARA PUSH DAN PULL PADA GITHUB

Diajukan untuk memenuhi tugas pengganti Mata Kuliah Aplikasi Berbasis Service

Dosen Pengampu: Andi Aljabar, S.Kom., M.TA



Disusun oleh:

Dede Ahmad Fauzan (NIM 24260014)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS TEKNIK DAN ILMU KOMPUTER

UNIVERSITAS NAHDLATUL ULAMA INDONESIA

2026

A. Informasi Umum

Keterangan	Isi
Mata Kuliah	Pemrograman Web
Topik	Alur kerja GitHub: push, pull, branch, dan penanganan konflik
Dosen Pengampu	Pak Andi Aljabar
Pertemuan	1
Tanggal Pertemuan	26 Oktober 2026
Durasi / Media	68 menit (via Zoom Meeting)

B. Konsep Dasar Git dan GitHub

Pengelolaan versi kode sumber dibutuhkan agar perubahan pada program dapat dilacak, dibatalkan, dan digabungkan ketika dikerjakan oleh lebih dari satu orang. Dua alat yang dipakai dalam praktik ini adalah Git dan GitHub, seperti dirangkum pada tabel berikut.

Istilah	Penjelasan
Git	Aplikasi kontrol versi yang berjalan di komputer lokal untuk mencatat riwayat perubahan berkas.
GitHub	Layanan hosting repositori Git di internet untuk menyimpan, berbagi, dan berkolaborasi.
Repository	Folder proyek beserta seluruh riwayat perubahannya.
Remote	Repositori di server (misalnya GitHub); nama standarnya `origin`.
Commit	Snapshot kondisi proyek pada satu titik waktu, disertai pesan penjelasan.
Branch	Cabang pengembangan; cabang utama umumnya bernama `main`.
Push	Mengirim commit dari repositori lokal ke GitHub.
Pull	Mengambil commit terbaru dari GitHub dan menggabungkannya ke repositori lokal.
Clone	Menyalin seluruh repositori dari GitHub ke komputer untuk pertama kali.

Setiap berkas dalam proyek berada pada salah satu dari tiga area, yaitu working directory (berkas yang sedang diedit), staging area (berkas yang siap di-commit), dan repository (riwayat commit). Perubahan harus melewati `git add` dan `git commit` di komputer lokal sebelum dikirim ke GitHub dengan `git push`. Untuk mengambil perubahan dari GitHub digunakan `git pull` atau `git clone`.

Push hanya diterima GitHub bila riwayat lokal merupakan kelanjutan langsung dari riwayat remote. Jika remote memiliki commit yang belum ada di lokal, push ditolak dan perubahan harus diambil terlebih dahulu dengan pull. Secara konsep, pull adalah gabungan dari fetch dan merge.

C. Persiapan dan Autentikasi

Setelah Git dipasang, versinya diperiksa dan identitas pengguna diatur satu kali. Alamat email sebaiknya sama dengan email akun GitHub agar commit terhubung ke profil.

```
git --version
git config --global user.name "Dede Ahmad Fauzan"
git config --global user.email "email@contoh.com"
git config --global init.defaultBranch main
git config --list
```

Sejak tahun 2021, GitHub tidak lagi menerima kata sandi akun untuk operasi Git melalui HTTPS. Terdapat tiga metode autentikasi pengganti.

Metode	Kelebihan	Kekurangan
Personal access token (HTTPS)	Mudah disiapkan	Token harus dijaga kerahasiaannya dan dapat kedaluwarsa
Kunci SSH	Tidak perlu memasukkan kredensial berulang kali	Persiapan awal lebih panjang
GitHub CLI (gh auth login)	Paling ringkas bagi pemula	Perlu memasang GitHub CLI

Apa pun metodenya, token dan kunci privat tidak boleh dimasukkan ke dalam repositori.

D. Praktik Push Pertama ke Repositori Kosong

Repositori kosong dibuat terlebih dahulu di GitHub, lalu diisi dari proyek lokal dengan urutan berikut.

No.	Perintah	Fungsi
1	`cd portfolio-dede`	Masuk ke folder proyek
2	`git init`	Membuat folder tersembunyi .git yang menyimpan riwayat dan konfigurasi
3	(membuat .gitignore)	Mendaftar berkas yang tidak boleh dilacak, misalnya .env dan node_modules/
4	`git status`	Menampilkan berkas yang belum dilacak, berubah, atau sudah di-staging
5	`git add .`	Memasukkan semua perubahan ke staging area

No.	Perintah	Fungsi
6	git commit -m "pesan"	Menyimpan snapshot dari staging ke riwayat lokal
7	git branch -M main	Memastikan nama branch adalah main
8	git remote add origin <url>	Mendaftarkan alamat repositori GitHub dengan nama origin
9	git push -u origin main	Mengirim commit ke GitHub dan menautkan main lokal dengan origin/main

Opsi -u (set upstream) menyimpan hubungan antara branch lokal dan branch remote, sehingga perintah berikutnya cukup git push dan git pull tanpa argumen tambahan.

E. Siklus Kerja Harian

Setelah push pertama, pekerjaan sehari-hari mengikuti siklus yang sama: pull, ubah berkas, periksa, commit, lalu push. Memulai dengan pull mengurangi kemungkinan push ditolak karena remote sudah lebih maju.

```
git pull
# ... edit berkas proyek ...
git status
git diff
git add .
git commit -m "feat: tambah deskripsi singkat"
git push
```

Commit sebaiknya kecil dan berfokus pada satu tujuan. Pesan commit dapat memakai prefiks sederhana berikut.

Jenis	Kegunaan	Contoh
feat	Fitur baru	feat: tambah halaman registrasi
fix	Perbaikan bug	fix: perbaiki validasi email
docs	Dokumentasi	docs: perbarui README
style	Perubahan tampilan tanpa mengubah logika	style: tambah mode gelap
refactor	Merapikan kode tanpa mengubah fungsi	refactor: pisahkan fungsi koneksi
chore	Tugas kecil atau konfigurasi	chore: tambah .gitignore

F. Pull, Fetch, dan Clone

git clone mengunduh seluruh repositori beserta riwayatnya ke komputer baru dan otomatis menetapkan remote origin. Perbedaan git fetch dan git pull ditunjukkan pada tabel berikut.

Aspek	git fetch	git pull
Mengunduh perubahan dari remote	Ya	Ya
Menggabungkan ke branch lokal	Tidak	Ya
Risiko mengubah berkas lokal	Tidak ada	Ada (dapat menimbulkan konflik)
Kapan digunakan	Ingin meninjau perubahan terlebih dahulu	Ingin langsung menyinkronkan

```
git clone https://github.com/USERNAME/portfolio-dede.git
git fetch origin
git log HEAD..origin/main --oneline
git merge origin/main

# atau langsung sinkron dengan riwayat yang lurus
git pull --rebase
```

Opsi `--rebase` meletakkan ulang commit lokal di atas commit terbaru dari remote sehingga riwayat berbentuk garis lurus. Rebase sebaiknya hanya dipakai pada commit lokal yang belum dibagikan kepada orang lain.

G. Branch dan Pull Request

Branch memungkinkan pengerjaan fitur tanpa mengganggu branch utama. Pull request (PR) adalah mekanisme GitHub untuk mengusulkan penggabungan sebuah branch ke `main` melalui peninjauan.

```
git switch -c fitur-login
git push -u origin fitur-login

# setelah pull request di-merge di GitHub
git switch main
git pull
git branch -d fitur-login
```

Alur pull request: (1) kirim branch fitur dengan `git push -u origin fitur-login`; (2) buka repositori di GitHub dan tekan *Compare & pull request*; (3) isi judul dan deskripsi perubahan; (4) setelah ditinjau dan disetujui, tekan *Merge pull request*; (5) sinkronkan kembali repositori lokal dengan `git switch main` dan `git pull`.

H. Penanganan Konflik

Konflik terjadi ketika dua perubahan memodifikasi baris yang sama pada berkas yang sama sehingga Git tidak dapat menggabungkannya secara otomatis. Git menandai bagian bermasalah dengan tiga penanda: bagian di antara `<<<<<<< HEAD` dan `=======` adalah versi lokal, sedangkan bagian di antara `=======` dan `>>>>>>> origin/main` adalah versi dari remote.

Langkah penyelesaian: pilih salah satu versi atau gabungkan keduanya, hapus ketiga penanda, tandai berkas selesai dengan `git add`, lalu buat commit.

```
git pull
# CONFLICT (content): Merge conflict in index.html

# setelah konflik diedit dan penanda dihapus
git add index.html
git commit -m "fix: selesaikan konflik pada index.html"
git push

# untuk membatalkan proses merge
git merge --abort
```

I. Studi Kasus: Website Portfolio

Prosedur di atas diterapkan pada pembuatan website portfolio statis atas nama Dede Ahmad Fauzan yang disimpan di repositori `portfolio-dede` dan dipublikasikan melalui GitHub Pages.

1. Struktur proyek

Berkas	Peran
`index.html`	Kerangka dan isi halaman: navigasi, hero, tentang, skill, proyek, dan kontak
`style.css`	Tampilan: variabel warna, tema gelap, tata letak grid dan flexbox, serta aturan responsif
`script.js`	Interaksi: pengganti tema, menu HP, efek ketik, animasi scroll, dan form kontak
`README.md`	Dokumentasi proyek

Berkas halaman utama harus bernama `index.html` karena GitHub Pages memuatnya otomatis sebagai halaman depan.

2. Pengujian lokal

Sebelum di-commit, halaman diuji dengan menjalankan server lokal.

```
python -m http.server 8000
```

Lalu buka `http://localhost:8000`. Hal yang diperiksa: kebenaran tautan menu, fungsi tombol tema, kerapian tampilan pada layar kecil (device toolbar peramban, tombol F12), dan tidak adanya pesan galat pada konsol.

3. Commit bertahap dan push

Pekerjaan dipecah menjadi empat commit kecil, masing-masing berisi satu tujuan.

```
git init
git remote add origin https://github.com/USERNAME/portfolio-dede.git

git add index.html
git commit -m "feat: tambah struktur halaman portfolio"

git add style.css
git commit -m "style: tambah tampilan responsif dan mode gelap"

git add script.js
git commit -m "feat: tambah interaksi tema, menu, dan animasi"

git add README.md
git commit -m "docs: tambah README"

git branch -M main
git push -u origin main
git log --oneline --graph
```

4. Publikasi dengan GitHub Pages

Langkah pengaktifan: (1) buka repositori di GitHub lalu pilih *Settings*; (2) pilih menu *Pages*; (3) pada *Build and deployment*, pilih *Source* berupa *Deploy from a branch*; (4) pilih branch main dan folder / (root), lalu tekan *Save*; (5) tunggu sekitar satu sampai tiga menit hingga alamat situs tampil. Alamat situs berpola https://<username>.github.io/<nama-repo>/. Setiap push baru ke main memicu pembaruan situs secara otomatis.

J. Pemecahan Masalah

Pesan galat / masalah	Penyebab	Solusi
fatal: not a git repository	Folder belum diinisialisasi	Jalankan git init atau pindah ke folder yang benar
remote origin already exists	Remote sudah pernah ditambahkan	git remote set-url origin <url>
Authentication failed	Memakai kata sandi akun	Gunakan token, kunci SSH, atau gh auth login
src refspec main does not match any	Belum ada commit, atau nama branch bukan main	Lakukan git commit, lalu git branch -M main
rejected (non-fast-forward)	Remote memiliki commit yang belum ada di lokal	git pull --rebase, lalu git push
refusing to merge unrelated histories	Riwayat lokal dan remote berasal dari awal yang	git pull origin main --allow-unrelated-histories

Pesan galat / masalah	Penyebab	Solusi
	berbeda	
Permission denied (publickey)	Kunci SSH belum terdaftar di GitHub	Tambahkan kunci publik pada Settings > SSH and GPG keys
.env atau token terlanjur ter-push	Berkas rahasia ikut di-commit	Segera cabut dan ganti kredensial, lalu hapus dari riwayat
CSS tidak termuat di GitHub Pages	Nama berkas berbeda huruf besar/kecil	Samakan penulisan nama berkas, misalnya git mv Style.css style.css
Halaman 404 pada alamat Pages	Berkas utama bukan index.html, atau build belum selesai	Pastikan index.html ada di direktori utama dan cek tab Actions

K. Praktik Terbaik

- Pull sebelum mulai bekerja dan sebelum push untuk meminimalkan konflik.
- Commit sering dengan ukuran kecil dan pesan yang jelas.
- Jangan pernah meng-commit data rahasia seperti kata sandi, token, kunci API, atau berkas .env; daftarkan pada .gitignore.
- Gunakan branch untuk fitur atau perbaikan agar main tetap stabil.
- Hindari git push --force pada branch bersama; jika terpaksa, gunakan --force-with-lease.
- Periksa git status sebelum setiap commit dan push.

L. Rencana Pertemuan Berikutnya

Pertemuan selanjutnya akan melanjutkan materi dengan mengonversi sketsa layout menjadi struktur HTML semantik, meliputi:

- Header, sidebar, dan navigasi
- Section, card, table, dan form
- Layout responsif, dengan acuan tampilan situs web pemerintahan seperti DKI Jakarta

M. Kesimpulan

Praktik ini memberikan gambaran lengkap tentang alur kerja GitHub, mulai dari persiapan dan autentikasi, push pertama ke repositori kosong, siklus kerja harian, pull, fetch, dan clone, hingga penggunaan branch, pull request, dan penanganan konflik. Studi kasus website portfolio menunjukkan bahwa prosedur yang sama berlaku dari push pertama hingga publikasi melalui GitHub Pages. Kebiasaan memperbarui kode secara berkala dengan pull, membuat commit kecil dengan pesan yang jelas, dan tidak pernah mengunggah data rahasia menjadi dasar kerja kolaboratif yang tertib.